ROUTING EN ANGULAR: NAVEGACIÓN, RUTAS Y GUARDS

SPA, RouterModule, definición de rutas, navegación, directivas, rutas activas, estrategias de precarga y guards

Apuntes revisados y ampliados
Documento elaborado a partir del fastbook original de Rodrigo Zavala

Contenido

Introducción: por qué importa el routing . . . 3

1. ¿Qué es una SPA? . . . 3

2. Módulo de enrutamiento de Angular . . . 3

2.1. Configuración de RouterModule . . . 4

2.2. Definición de rutas . . . 4

2.3. Propiedades adicionales de una ruta . . . 5

2.4. Router outlet . . . 7

3. Navegación . . . 7

3.1. Inyección del servicio Router . . . 7

3.2. Navegación a una ruta . . . 7

3.3. Navegación con parámetros . . . 8

3.4. Rutas absolutas y relativas . . . 8

4. Directivas de navegación . . . 8

4.1. routerLink . . . 8

4.2. routerLinkActive . . . 9

5. Rutas activas . . . 9

6. Estrategias de precarga . . . 10

6.1. PreloadAllModules . . . 10

6.2. Precarga selectiva (estrategia personalizada) . . . 10

7. Guards . . . 11

7.1. CanActivate . . . 11

7.2. CanActivateChild . . . 12

7.3. CanDeactivate . . . 12

7.4. Resolve . . . 12

8. Qué hemos aprendido . . . 13

Introducción: por qué importa el routing

Este fastbook explica cómo funciona el **routing de Angular**: desde la configuración más básica hasta rutas complejas, cómo **pasar información entre vistas** y cómo **proteger el acceso a las rutas con guards**.

Angular ya trae los módulos, clases y servicios necesarios para gestionar la navegación, así que no hay que programar ese sistema a mano: basta con configurarlo. Esto ahorra mucho trabajo en tareas que se repiten a diario en cualquier aplicación.

1. ¿Qué es una SPA?

Una **SPA (single page application)** es una aplicación web que **carga una única página HTML** y, a medida que el usuario interactúa, **reemplaza dinámicamente el contenido** de esa misma página en lugar de pedir páginas completas nuevas al servidor. Sus características principales:

Característica	Qué significa
Carga inicial única	Al entrar se carga un solo HTML con la estructura base (cabecera, pie y un contenedor principal). Las webs tradicionales cargan un HTML nuevo en cada interacción.
Navegación sin recarga	Con JavaScript se piden datos al servidor y se actualiza solo el área principal: sin interrupciones visibles, experiencia más rápida y fluida.
Manejo de rutas	Un **router** gestiona las URL internas: visitar */products* muestra la lista de productos sin recargar la página.
Interacción asíncrona	Uso intensivo de peticiones asíncronas (Fetch API, Promises) para obtener datos o ejecutar acciones.
Estado dinámico	Los componentes cambian su contenido según las acciones del usuario o los datos recibidos, sin recargar.
Rendimiento	La estructura se carga una vez y luego se actualiza de forma incremental: menos latencia percibida.
Frameworks	Angular, React o Vue.js facilitan el estado, la navegación y el enrutamiento.

> **★ Lo más importante**
> - SPA = un único HTML cuyo contenido se sustituye dinámicamente, sin recargar la página.
> - El router traduce cada URL interna en la vista que hay que mostrar.
> - Ventaja clave: navegación rápida y fluida, con menos latencia percibida.

2. Módulo de enrutamiento de Angular

Como Angular construye SPAs, la aplicación navega entre vistas o componentes en función de la URL en lugar de cargar páginas nuevas. Quien hace posible esto es el **RouterModule** (módulo de enrutamiento).

2.1. Configuración de RouterModule

Se importa `RouterModule` y el tipo `Routes` desde `@angular/router` y se registra en el módulo principal (normalmente `app.module.ts`):

- Se crea un array `routes` con las definiciones de ruta.
- En `@NgModule` se importa `RouterModule.forRoot(routes)` para las rutas principales. `forRoot` **se usa una sola vez** (en la raíz); en módulos de rutas hijas se usa `forChild`.

```
import { RouterModule, Routes } from '@angular/router';

const routes: Routes = [
  // Definir rutas aquí
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: []
})
export class AppModule { }
```

Es muy habitual sacar las rutas a su propio archivo: un `AppRoutingModule` que importa `RouterModule.forRoot(routes)` y a la vez **exporta** `RouterModule`, de modo que `AppModule` solo tiene que importar `AppRoutingModule`.

```
/******** app-routing.module.ts ********/
@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }

/******** app.module.ts ********/
@NgModule({ imports: [AppRoutingModule] })
export class AppModule { }
```

Nota práctica (no aparece en el fastbook): *en aplicaciones standalone (sin módulos, Angular 14+) no se usa* `RouterModule.forRoot`*, sino* `provideRouter(routes)` *en* `bootstrapApplication` *(o en* `app.config.ts`*).*

2.2. Definición de rutas

Cada elemento del array es un objeto `Route` que relaciona **una URL con un componente o módulo**. Las dos propiedades básicas son:

- `path`: la parte de la URL que activa la ruta (`'about'` → */about*).
- `component`: el componente que se carga cuando la ruta se activa (`AboutComponent`).

```
const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'about', component: AboutComponent },
  { path: 'contact', component: ContactComponent },
];
```

2.3. Propiedades adicionales de una ruta

Además de `path` y `component`, una ruta admite otras propiedades que se usan según lo que necesite la aplicación:

Propiedad	Para qué sirve
`redirectTo`	Redirige automáticamente a otra ruta. Útil para rutas por defecto o comodín (`'**'`).
`pathMatch`	Cómo se compara el path: `'prefix'` (por defecto) o `'full'` (la URL debe coincidir entera).
`children`	Rutas anidadas: diseños complejos y jerarquías de rutas.
`loadChildren`	**Lazy loading**: carga un módulo solo cuando se necesita.
`canActivate`	Guards que se evalúan **antes de entrar** en la ruta (p. ej. autenticación).
`canActivateChild`	Igual que canActivate, pero para las **rutas hijas**.
`canDeactivate`	Comprueba si se puede **salir** de la ruta (cambios sin guardar).
`canMatch`	Decide si la ruta **puede emparejarse**; si no, el router prueba la siguiente.
`resolve`	**Carga datos antes** de activar la ruta (p. ej. desde una API).
`data`	Datos estáticos personalizados accesibles desde el componente (título, flags…).
`outlet`	En qué `<router-outlet>` con nombre se pinta la ruta si hay varios.

redirectTo y pathMatch

```
const routes: Routes = [
  { path: '', redirectTo: 'home', pathMatch: 'full' },  // ruta por defecto
  // ... resto de rutas ...
  { path: '**', redirectTo: 'home' },                    // comodín: siempre la última
];
```

Con `'prefix'` la ruta coincide si la URL **empieza por** ese path (segmento a segmento); con `'full'` la URL debe coincidir **completa**.

Nota práctica (no aparece en el fastbook): *el fastbook dice que prefix busca coincidencias «en cualquier parte de la URL»; en realidad compara desde el principio. Por eso una redirección desde `''` necesita `pathMatch: 'full'`: con prefix, la ruta vacía es prefijo de todas las URL y redirigiría siempre. Y el comodín `'**'` debe ir al final, porque el router recorre las rutas en orden y se queda con la primera que coincide.*

children: rutas anidadas

```
{
  path: 'products',
  component: ProductsComponent,
  children: [
    { path: 'details/:id', component: ProductDetailComponent },
    { path: 'edit/:id',    component: ProductEditComponent },
  ],
}
```

El componente padre (`ProductsComponent`) necesita su propio `<router-outlet>` donde se pintan los hijos (*/products/details/5*, */products/edit/5*).

loadChildren: lazy loading

```
{
  path: 'lazy',
  loadChildren: () => import('./lazy/lazy.module').then(m => m.LazyModule),
  data: { title: 'Módulo Perezoso' },
}
```

La carga perezosa aporta mucha flexibilidad: **reduce el tamaño del paquete inicial** (la app arranca antes) y obliga a una organización **más modular y mantenible**. En apps medianas y grandes es lo habitual, aunque se puede mezclar: una pantalla sencilla como *Home* puede cargarse directamente con `component`.

Nota práctica (no aparece en el fastbook): *con componentes standalone existe* `loadComponent: () => import('./x.component').then(m => m.XComponent)` *para cargar de forma perezosa un único componente, sin módulo.*

canActivate, canActivateChild y canDeactivate

```
{ path: 'about', component: AboutComponent, canActivate: [AuthGuard] },

{ path: 'products', component: ProductsComponent,
  children: [ /* details, edit... */ ],
  canActivateChild: [ProductChildGuard] },

{ path: 'details/:id', component: ProductDetailComponent,
  canDeactivate: [ProductDetailDeactivate] },
```

canMatch

Si el guard de `canMatch` devuelve `false`, el router **ignora esa ruta y sigue buscando** otra definición que coincida con la URL; si no encuentra ninguna, acaba en el comodín `'**'` (o da error de match si no existe).

```
export const authGuard: CanMatchFn = (route, segments) => false;

export const routes: Routes = [
  { path: '', pathMatch: 'full', redirectTo: 'users' },
  { path: 'users', canMatch: [authGuard], component: UsersComponent },
  { path: '**', component: NotFoundComponent },  // aquí acaba si canMatch = false
];
```

Ventaja de canMatch: se pueden tener **dos rutas con el mismo path** (por ejemplo dos */dashboard*, una para *admin* y otra para *users*). La primera lleva un `canMatch` que comprueba si es admin; si no lo es, el router salta a la segunda, con el componente de usuarios. La URL es la misma para ambos.

resolve, data y outlet

```
{ path: 'details/:id', component: ProductDetailComponent,
  resolve: { productData: ProductResolve } },

export const ProductResolve: ResolveFn<Product> =
  (route: ActivatedRouteSnapshot, state: RouterStateSnapshot) =>
    inject(ProductService).getProduct(route.paramMap.get('id')!);

{ path: 'home',  component: HomeComponent,  data: { title: 'Inicio' } },
{ path: 'admin', component: AdminComponent, outlet: 'adminOutlet' },
```

`ProductResolve` es una constante de tipo `ResolveFn<T>` con una función lambda que devuelve el producto según el ID de la URL. Así **los datos ya están disponibles cuando se pinta la vista**.

2.4. Router outlet

En la plantilla principal (normalmente `app.component.html`) se coloca la directiva `<router-outlet>`: un hueco vacío donde Angular **renderiza dinámicamente el componente de la ruta activa**. Es «la magia» que permite navegar sin recargar la página: cabecera y pie se quedan fijos y solo cambia el contenido.

```
<header></header>
<div class="body">
  <router-outlet></router-outlet>
</div>
<footer></footer>
```

> **★ Lo más importante**
>
> - `RouterModule.forRoot(routes)` una sola vez en la raíz; `forChild` en módulos hijos.
> - Cada ruta une un `path` con un `component` (o un módulo con `loadChildren`).
> - `redirectTo` desde `''` necesita `pathMatch: 'full'`; el comodín `'**'` va siempre el último.
> - `loadChildren` = lazy loading: paquete inicial más pequeño y app más modular.
> - `canMatch` permite dos rutas iguales: si el guard falla, el router prueba la siguiente.
> - `resolve` carga datos antes de mostrar la vista; `<router-outlet>` es donde se pinta la ruta.

3. Navegación

Hay varias formas de lanzar la navegación desde código, todas apoyadas en servicios que ya trae el framework.

3.1. Inyección del servicio Router

El servicio `Router` (de `@angular/router`) ofrece los métodos para navegar a las rutas declaradas. Como cualquier servicio, se **inyecta en el constructor o con** `inject()`.

```
import { Router } from '@angular/router';

constructor(private router: Router) { }
// o bien:  private router = inject(Router);
```

3.2. Navegación a una ruta

Se navega con el método `navigate()`, normalmente en respuesta a un clic, una opción de menú o cualquier otro evento:

```
goToAboutPage() {
  this.router.navigate(['/about']);   // carga AboutComponent en el router-outlet
}
```

3.3. Navegación con parámetros

Para **pasar datos a través de la ruta** se declara un parámetro en el path (`product/:id`) y se le da valor al navegar:

```
// Configuración
{ path: 'product/:id', component: ProductDetailComponent }

// Navegar pasando el ID  ->  /product/42
goToProductDetail(productId: number) {
  this.router.navigate(['/product', productId]);
}

// Leerlo en ProductDetailComponent con ActivatedRoute
constructor(private route: ActivatedRoute) { }
ngOnInit() {
  this.productId = +this.route.snapshot.paramMap.get('id')!;  // + convierte a number
}
```

Nota práctica (no aparece en el fastbook): *el fastbook llama QueryParams a los parámetros tipo* `:id`*, pero son* ***parámetros de ruta*** *(forman parte del path). Los query params son los que van tras* `?` *(/products?page=2): se envían con* `navigate(['/products'], { queryParams: { page: 2 } })` *y se leen con* `route.snapshot.queryParamMap`.

3.4. Rutas absolutas y relativas

Tipo	Cómo se escribe	Ejemplo
Absoluta	**Empieza por** `/`: se resuelve desde la raíz.	`this.router.navigate(['/about'])`
Relativa	**Sin** `/` **inicial**; se indica respecto a qué ruta con `relativeTo`.	`navigate(['../other'], { relativeTo: this.route })`

> **★ Lo más importante**
>
> - `Router` se inyecta (constructor o `inject`) y se navega con `navigate([...])`.
> - Parámetros de ruta: se declaran como `:id`, se pasan como elemento del array y se leen con `ActivatedRoute`.
> - Absoluta = empieza por `/`; relativa = sin `/` + `relativeTo`.

4. Directivas de navegación

También se puede navegar **directamente desde la plantilla** con las directivas que aporta `RouterModule` (en standalone, importando `RouterLink` y `RouterLinkActive`).

4.1. routerLink

Convierte cualquier elemento (<a>, botón…) en un **enlace de navegación interna** sin recargar la página.

```
<a routerLink="/home">Inicio</a>
<a routerLink="/products">Productos</a>
<button [routerLink]="['/dashboard', 1]">Ir al Dashboard</button>
```

- Los dos <a> enlazan con */home* y */products* usando un string fijo.
- El botón usa `[routerLink]` **con array** (property binding) para construir la ruta dinámicamente: */dashboard/1*.

4.2. routerLinkActive

Añade una clase CSS cuando la ruta del enlace está activa, ideal para resaltar la opción actual del menú.

```
<a routerLink="/home" routerLinkActive="active">Inicio</a>
<a routerLink="/products" routerLinkActive="active">Productos</a>
```

Nota práctica (no aparece en el fastbook): *como la comparación es por prefijo, un enlace a `/` estaría siempre activo. Se evita con* `[routerLinkActiveOptions]="{ exact: true }"`. *Además, en un* <a> *usa* `routerLink` *y no* `href`*:* `href` *recarga la página completa y se pierde la ventaja de la SPA.*

> **★ Lo más importante**
> - `routerLink` navega desde la plantilla; `[routerLink]="['/ruta', param]"` para rutas dinámicas.
> - `routerLinkActive="clase"` aplica la clase cuando esa ruta está activa.

5. Rutas activas

Se puede **consultar la información de la ruta activa** (parámetros, datos resueltos, etc.) a través de tres objetos que ofrecen vistas distintas:

Objeto	Naturaleza	Dónde se usa
`ActivatedRoute`	Versión **activa y mutable**: expone parámetros, datos y rutas hijas (también como Observables).	Componentes y servicios que necesitan la ruta actual.
`ActivatedRouteSnapshot`	**Foto inmutable** de la ruta en el momento de la activación.	**Guards** y resolvers, antes de que se pinte la vista.
`RouterStateSnapshot`	Foto inmutable del **estado completo** del router: todas las rutas activas y la URL.	Historial de navegación, lógica de redirección (`state.url`).

```
// ActivatedRoute (componente)
constructor(private route: ActivatedRoute) {
  const id = this.route.snapshot.params['id'];
}

// ActivatedRouteSnapshot y RouterStateSnapshot (guard)
canActivate(route: ActivatedRouteSnapshot, state: RouterStateSnapshot): boolean {
  const id = route.params['id'];      // parámetro en el momento de la activación
  const currentUrl = state.url;       // URL a la que se intenta ir
  ...
}
```

Nota práctica (no aparece en el fastbook): `route.snapshot` *solo se lee una vez. Si se navega de /product/1 a /product/2, Angular reutiliza el mismo componente y el snapshot no se actualiza; en ese caso hay que suscribirse:* `this.route.paramMap.subscribe(p => this.load(p.get('id')))`.

> ★ **Lo más importante**
> - `ActivatedRoute`: la ruta actual, viva (Observables) — para componentes.
> - `ActivatedRouteSnapshot`: foto de una ruta en la activación — típico en guards/resolvers.
> - `RouterStateSnapshot`: foto de todo el estado del router, incluida la URL.

6. Estrategias de precarga

La precarga consiste en **descargar en segundo plano los módulos lazy** mientras el usuario usa la aplicación, de modo que cuando navega a ellos ya están listos. Mejora la **velocidad de las rutas y la experiencia del usuario**, sobre todo en aplicaciones grandes con módulos pesados.

6.1. PreloadAllModules

Tras cargar la página principal, Angular **descarga todos los módulos lazy de forma asíncrona**, aunque el usuario no haya visitado esas rutas. Se activa con la opción `preloadingStrategy` de `forRoot()`:

```
import { RouterModule, Routes, PreloadAllModules } from '@angular/router';

@NgModule({
  imports: [RouterModule.forRoot(routes, { preloadingStrategy: PreloadAllModules })],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

Cuidado: en aplicaciones grandes puede descargar módulos que nunca se usan y **penalizar el rendimiento**.

6.2. Precarga selectiva (estrategia personalizada)

Permite **elegir qué módulos precargar**. Se marca cada ruta con una propiedad en `data` y se crea una clase que implementa `PreloadingStrategy`, cuyo método `preload` decide ruta a ruta:

```
const routes: Routes = [
  { path: 'dashboard', loadChildren: () => import('./dashboard/dashboard.module')
      .then(m => m.DashboardModule), data: { preload: true } },   // se precarga
  { path: 'profile', loadChildren: () => import('./profile/profile.module')
      .then(m => m.ProfileModule), data: { preload: false } },    // no se precarga
];

@Injectable({ providedIn: 'root' })
export class PreloadSelectedModules implements PreloadingStrategy {
  preload(route: Route, load: () => Observable<any>): Observable<any> {
    return route.data?.['preload'] ? load() : of(null);
  }
}

RouterModule.forRoot(routes, { preloadingStrategy: PreloadSelectedModules })
```

Nota práctica (no aparece en el fastbook): *el fastbook presenta* `PreloadSelectedModules` *como si fuera una estrategia de Angular, pero* ***no lo es****: es una clase propia (el nombre es libre). Las que trae el framework son* `NoPreloading` *(la de por defecto) y* `PreloadAllModules`*. La clase necesita* `@Injectable` *para poder registrarse. En standalone:* `provideRouter(routes, withPreloading(PreloadAllModules))`.

> **★ Lo más importante**
>
> - Precarga = descargar módulos lazy en segundo plano antes de que el usuario los pida.
> - `PreloadAllModules`: todo, de golpe; sencillo pero arriesgado en apps grandes.
> - Selectiva: clase propia que implementa `PreloadingStrategy` + flag en `data`.
> - Por defecto Angular no precarga nada (`NoPreloading`).

7. Guards

Los **guards** controlan el acceso a rutas o componentes: permiten **proteger rutas y decidir según la lógica de negocio** (usuario autenticado, permisos, confirmar antes de salir con cambios sin guardar…). Pueden acceder a los parámetros de `ActivatedRouteSnapshot` y `RouterStateSnapshot`.

Guard	Cuándo se ejecuta	Uso típico
`CanActivate`	Antes de entrar en una ruta.	Comprobar si el usuario está autenticado.
`CanActivateChild`	Antes de entrar en cualquier ruta hija.	Proteger de una vez una sección entera.
`CanDeactivate`	Antes de salir de una ruta.	Avisar de cambios sin guardar.
`CanMatch`	Al intentar emparejar la ruta.	Rutas alternativas con el mismo path (admin/users).
`Resolve`	Antes de activar la ruta.	Precargar los datos que necesita la vista.

7.1. CanActivate

Devuelve `true` para dejar pasar o un `UrlTree` para **redirigir** (por ejemplo, al login):

```
@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  constructor(private router: Router) {}

  canActivate(route: ActivatedRouteSnapshot, state: RouterStateSnapshot): boolean | UrlTree {
    const isAuthenticated = /* lógica de autenticación */;
    return isAuthenticated ? true : this.router.createUrlTree(['/login']);
  }
}

const routes: Routes = [
  { path: 'dashboard', component: DashboardComponent, canActivate: [AuthGuard] },
];
```

Nota práctica (no aparece en el fastbook): *desde Angular 15.2 los guards basados en clases (`implements CanActivate`) están **deprecados** en favor de los **guards funcionales**, que es como se escriben hoy:*

```
export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  return auth.isLoggedIn() ? true : inject(Router).createUrlTree(['/login']);
};
```

7.2. CanActivateChild

Como `CanActivate`, pero se aplica a las **rutas hijas**. En el ejemplo del fastbook, la ruta padre *protected* agrupa *dashboard* y *profile*:

```
{
  path: 'protected',
  canActivateChild: [AuthGuard],   // se comprueba al entrar en cada hija
  children: [
    { path: 'dashboard', component: DashboardComponent },
    { path: 'profile',   component: ProfileComponent },
  ],
}
```

Nota práctica (no aparece en el fastbook): *el código del fastbook usa* `canActivate` *en la ruta padre, lo que solo protege la entrada a la sección; para que se evalúe en cada navegación entre hijas hay que usar* `canActivateChild`*, como arriba.*

7.3. CanDeactivate

Se ejecuta **antes de abandonar la ruta**. Hoy la forma recomendada es definirlo **directamente en la ruta**, recibiendo la **instancia del componente** y usando su lógica:

```
{
  path: 'example',
  component: ExampleComponent,
  canDeactivate: [(component: ExampleComponent) => !component.hasUnsavedChanges],
}
```

7.4. Resolve

Los **resolvers** precargan datos antes de mostrar el componente, garantizando que la vista tenga lo que necesita al pintarse. Versión con clase (la funcional `ResolveFn` está en el apartado 2.3):

```
@Injectable({ providedIn: 'root' })
export class ProductResolve implements Resolve<Product> {
  constructor(private productService: ProductService) {}

  resolve(route: ActivatedRouteSnapshot, state: RouterStateSnapshot):
      Observable<Product> | Promise<Product> | Product {
    return this.productService.getProductById(route.params['id']);
  }
}
```

Nota práctica (no aparece en el fastbook): *el dato resuelto se lee en el componente con* `this.route.snapshot.data['productData']` *(la clave es la que se puso en* `resolve: { productData: ... }`*).*

★ Lo más importante

- Guards = control de acceso a rutas según la lógica de negocio.
- `CanActivate` (entrar) · `CanActivateChild` (hijas) · `CanDeactivate` (salir) · `CanMatch` (emparejar) · `Resolve` (datos previos).
- Pueden devolver `boolean`, un `UrlTree` para redirigir, o un Observable/Promise de ellos.
- Hoy se escriben como funciones (`CanActivateFn`, `ResolveFn`…) con `inject()`.

8. Qué hemos aprendido

Hemos visto qué es una **SPA** y por qué el **RouterModule** es la pieza que permite navegar entre vistas sin recargar la página; cómo configurarlo con `forRoot`/`forChild` y definir rutas con `path`, `component` y propiedades como `redirectTo`, `pathMatch`, `children`, `loadChildren`, `resolve` o `data`; dónde se pintan con `<router-outlet>`; cómo **navegar** desde código con `Router.navigate()` (con parámetros, rutas absolutas y relativas) y desde la plantilla con `routerLink` y `routerLinkActive`; cómo **leer la ruta activa** con `ActivatedRoute` y sus snapshots; cómo **precargar módulos lazy** con `PreloadAllModules` o una estrategia propia; y cómo **proteger rutas con guards**.

Con esto, Tea Minder ya puede tener varias pantallas navegables y, más adelante, rutas protegidas para la parte de autenticación y el CRUD de tés.

¡Enhorabuena! Documento completado